



C#

Level 1

Anonymous Type



C#

Level 1

Anonymous Type

Definition

Anonymous Type

 An anonymous type is a type created automatically by the compiler without explicitly giving it a type name.

 It is commonly created using ``var`` with the ``new`` keyword.

Explanation

Creating an Anonymous Type

-  Properties are defined directly inside the object initializer.
-  The compiler automatically determines the property's types.

Procedures

- 1 Create an object using ``new``.
- 2 Define its properties inside ``{ }``.
- 3 Use ``var`` to store the anonymous object.
- 4 Access properties using the ``.`` operator.
- 5 Use nested properties through the same dot notation.

💡 Example



```
var student = new
{
    Id = 20,
    FirstName = "Dania",
    LastName = "Aljaseem"
};
Console.WriteLine(student.Id);
Console.WriteLine(student.FirstName);
Console.WriteLine(student.LastName);
```

➡ Result

20

Dania

Aljaseem

💡 Example



```
var student = new
{
    Id = 21,
    FirstName = "Hadi",
    Address = new
    {
        City = "Perstorp",
        Country = "Sweden"
    }
};
Console.WriteLine(student.Address.City);
Console.WriteLine(student.Address.Country);
```

➡ Result

21

Hadi

Aljasem

1

Perstorp

Sweden

Note

- ◆ An anonymous type does not have a name that you define.
- ◆ Its properties are strongly typed and determined by the compiler.
- ◆ The complete object can also be printed directly:
`Console.WriteLine(student);`

⊘ Warning

◆ Anonymous type properties cannot be reassigned after creation.

▮ Invalid:

```
student.Id = 21;
```

◆ Anonymous types are mainly useful for temporary data structures and are commonly used in LINQ queries.



Comparison

▨ Named Type

- ✓ Has an explicitly defined type name.
- ✓ Can be reused throughout the program.
- ✓ Suitable for data that needs a defined structure.

▤ Anonymous Type

- ✓ No type name is explicitly defined.
- ✓ Created directly with ``new { }``.
- ✓ Properties are read-only.
- ✓ Suitable for temporary data.

✦ Summary

- ✓ Anonymous types are created without explicitly defining a type name.
- ✓ ``var`` stores the automatically generated anonymous type.
- ✓ Properties are defined inside ``new { }``.
- ✓ Properties are read-only after creation.
- ✓ Nested anonymous types are supported.
- ✓ Properties are accessed using the ``.`` operator.

Thank you 😊